МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт № 8 «Компьютерные науки»

ЛАБОРАТОРНАЯ РАБОТА № 2

по курсу «Информационный поиск»

«Поисковый робот»

Выполнил: Д. А. Кулешов
Группа: М8О-408Б-23
Преподаватель: А. А. Кухтичев
Отметка о сдаче: _____
Оценка: _____

Москва 2026

1 Задание

Цель работы — разработать поискового робота для массовой загрузки документов из выбранных в первой лабораторной источников и сохранить исходные документы в базе данных для последующей обработки и индексации.

В соответствии с заданием поисковый робот должен:

- запускаться с единственным позиционным аргументом — путём к YAML-конфигурации;
- получать параметры базы данных из секции `db`;
- получать задержки и другие параметры обхода из секции `logic`;
- сохранять нормализованный URL, сырой HTML, название источника и время загрузки в формате Unix timestamp;
- корректно останавливаться в любой момент;
- после повторного запуска продолжать обход с сохранённой позиции;
- периодически проверять ранее загруженные документы и обновлять их только при изменении содержимого.

В работе обрабатываются три источника технических документов: Wikipedia, arXiv HTML и Microsoft Learn. Документы хранятся в MongoDB. Для выкачки разрешено использовать интерпретируемый язык, поэтому робот реализован на Python.

2 Краткое описание метода решения

Робот построен как асинхронная программа. Для трёх источников одновременно запускаются независимые задачи `asyncio`, но запросы к одному домену выполняются последовательно с собственной задержкой. Такой подход позволяет не ждать arXiv, когда доступна Wikipedia или Microsoft Learn, и одновременно не создавать избыточную нагрузку на один сайт.

Общая последовательность обработки документа следующая:

1. обнаружить идентификатор и URL через API, категории или sitemap;
2. нормализовать URL и проверить `robots.txt`;
3. проверить, требуется ли повторная загрузка;
4. выполнить HTTP-запрос с ограничением скорости и размера ответа;

5. вычислить SHA-256 сырого HTML;
6. добавить новый документ, обновить изменившийся либо только отметить время проверки неизменившегося;
7. сохранить позицию обхода в MongoDB.

MongoDB используется не только для самих HTML-документов, но и для очереди Wikipedia и позиций обхода arXiv и Microsoft Learn. Поэтому после остановки не требуется повторно начинать обход с первого URL.

3 Журнал выполнения задания

3.1 Конфигурация и запуск

Единственный аргумент программы задаётся через `argparse`:

```
parser = argparse.ArgumentParser(description="Поисковый робот MAI-IR")
parser.add_argument("config", type=Path, help="путь к YAML-конфигурации")
arguments = parser.parse_args()
config = load_config(arguments.config)
raise SystemExit(asyncio.run(run(config)))
```

Рабочая конфигурация находится в `config.yaml`. Основные параметры приведены в таблице 1.

Таблица 1 — Основные параметры поискового робота

Параметр	Значение	Назначение
`db.uri`	`mongodb://localhost:27017`	подключение к MongoDB
`db.database`	`mai_ir`	имя базы данных
`logic.request_timeout_seconds`	60	общий тайм-аут HTTP-запроса
`logic.retries`	4	число повторов временной ошибки
`logic.backoff_seconds`	2	основа экспоненциальной паузы
`logic.max_response_bytes`	14 000 000	предельный размер HTML
`logic.connections_per_host`	1	одно соединение на домен
`logic.lease_seconds`	300	срок владения задачей очереди
`logic.recrawl_after_seconds`	604 800	повторная проверка через 7 суток
`delay_seconds`	1 / 3 / 1 с	Wikipedia / arXiv / Microsoft Learn

Конфигурация проверяется до подключения к сети. Обязательны секции `db`, `logic` и `sources`, адрес базы, имя базы, задержки для каждого включённого источника и контакт в User-Agent. В User-Agent указан адрес `workdimila@gmail.com`, чтобы администратор сайта мог связаться со мной при проблемах.

Для запуска используются команды:

```
uv sync
docker compose up -d
uv run python src/crawler/main.py config.yaml
```

Контейнер MongoDB доступен только на `127.0.0.1:27017`, поэтому база не публикуется во внешнюю сеть.

3.2 HTTP-слой и ограничение нагрузки

Перед обходом каждого домена робот загружает `robots.txt`. Если файл невозможно проверить, источник не обходится: в этом случае безопаснее остановить источник, чем предположить разрешение.

Объект `RateLimiter` хранит время последнего запроса и защищён асинхронной блокировкой. Даже если внутри источника появится несколько корутин, они не смогут одновременно обойти установленную задержку.

```
async with self._lock:
    remaining = self.delay_seconds - (
        time.monotonic() - self._last_request
    )
    if remaining > 0:
        await wait_or_stop(stop_event, remaining)
    self._last_request = time.monotonic()
```

Для статусов 429, 500, 502, 503 и 504 применяется экспоненциальная пауза с небольшим случайным `jitter`. Если сервер прислал `Retry-After`, используется его значение. Пауза прерывается событием остановки, поэтому нажатие `Ctrl+C` не заставляет ждать окончания длинного backoff. Ответ читается блоками по 64 KiB; после превышения 14 000 000 байт документ пропускается, но источник продолжает работу.

В конфигурации разрешено шесть соединений суммарно и одно соединение на хост. Фактически три источника могут работать параллельно, а внутри каждого домена запросы остаются последовательными. Это даёт ускорение относительно полностью последовательной выкачки без агрессивного параллелизма и риска блокировки.

3.3 Нормализация и хранение документов

Нормализация URL приводит схему и домен к нижнему регистру, удаляет фрагмент, стандартный порт, повторные и завершающие косые черты, а также

параметры отслеживания `utm_*`, `fbclid`, `gclid` и другие. Оставшиеся параметры сортируются. Таким образом, адреса, отличающиеся только рекламной меткой или порядком query- параметров, представляются одинаково.

Для коллекции `documents` созданы два уникальных индекса:

```
await documents.create_index(
    [("source", ASCENDING), ("source_id", ASCENDING)],
    unique=True,
)
await documents.create_index(
    [("source", ASCENDING), ("url", ASCENDING)],
    unique=True,
)
```

Первый индекс запрещает повтор одного документа источника, второй — повтор нормализованного URL внутри источника. Поля MongoDB приведены в таблице 2.

Таблица 2 — Схема сырого документа в MongoDB

Поле	Тип	Содержание
`source`	строка	название источника
`source_id`	строка	устойчивый идентификатор источника
`url`	строка	нормализованный URL
`title`	строка	заголовок документа
`raw_html`	строка UTF-8	полный сырой HTML
`raw_size`	целое	размер HTML в байтах UTF-8
`metadata`	объект	поля конкретного источника
`content_hash`	строка	SHA-256 сырого HTML
`etag`	строка или null	значение HTTP ETag
`last_modified`	строка или null	HTTP Last-Modified
`first_fetched_at`	Unix timestamp	время первой загрузки
`fetched_at`	Unix timestamp	время загрузки текущей версии
`checked_at`	Unix timestamp	время последней проверки

Поле `first_fetched_at` устанавливается только при вставке. Если HTML изменился, обновляются `raw_html`, `content_hash`, `fetched_at` и `checked_at`. Если хеш совпадает, исходный HTML повторно не записывается и меняется только `checked_at`.

3.4 Обход Wikipedia

Обход начинается с семи тематических категорий: компьютерные науки, программная инженерия, искусственный интеллект, информационный поиск, базы данных, сети и компьютерная безопасность. MediaWiki API возвращает страницы и подкатегории. Подкатегории раскрываются до глубины 6.

Для Wikipedia используется постоянная очередь `crawl_queue`. Задача содержит тип `category` или `page`, ключ, глубину, приоритет, статус и время следующей попытки. Взятие задачи выполняется атомарно через `find_one_and_update`. На время обработки устанавливается lease. Если процесс завершился аварийно, после истечения lease задача снова может быть получена.

После успешной обработки задача получает статус `done`. При `Ctrl+C` текущая задача возвращается в `pending`, поэтому повторный запуск продолжает с неё, а не создаёт очередь заново. Уникальный индекс `(source, kind, key)` не позволяет добавить одну страницу или категорию дважды.

3.5 Обход arXiv

Метаданные arXiv читаются из Atom API, а сырой документ — со страницы `https://arxiv.org/html/<id>`. Выбраны категории `cs.IR`, `cs.AI`, `cs.CL`, `cs.DB`, `cs.DC`, `cs.CR`, `cs.LG` и `cs.SE`.

Первоначальный вариант с глобальным параметром `start` переставал работать около смещения 10 000 и постоянно получал HTTP 500. Поэтому обход разбит на месячные диапазоны `submittedDate`. В `crawl_state` после каждой статьи сохраняются категория, месяц, смещение страницы и индекс элемента внутри ответа. Повторный запуск продолжает тот же месяц с конкретной записи.

Для повторной проверки старых статей сначала одним POST-запросом проверяются метаданные до 100 arXiv ID. Если поле `updated` не изменилось, HTML не скачивается. Если API возвращает ошибку для пакета, пакет делится пополам. Единичный проблемный ID откладывается и не останавливает другие документы.

3.6 Обход Microsoft Learn

Сначала загружается `sitemapindex.xml`, затем выбираются только англоязычные sitemap с `_en-us_`. Исключаются ответы пользователей, видео, события, профили, сертификаты, API-справочник и старые версии. Для каждой sitemap обрабатываются подходящие адреса `/en-us/`.

После каждого URL в `crawl_state` сохраняются текущая sitemap и индекс следующего URL. Если робот остановлен, повторный запуск продолжает с сохранённой позиции. Для уже известного документа сначала сравнивается `lastmod` из sitemap. При совпадении HTML не загружается, обновляется только `checked_at`.

Состояние продолжения для трёх источников приведено в таблице 3.

Таблица 3 — Состояние продолжения обхода

Источник	Состояние	Момент сохранения
Wikipedia	задачи `pending`, `processing`, `done`, lease	после каждой категории или страницы
arXiv	категория, месяц, `start`, `entry_index`	после каждой статьи и страницы Atom API
Microsoft Learn	`sitemap_url`, `url_index`	после каждого URL и завершения sitemap

3.7 Остановка и параллельное выполнение источников

Обработчики `SIGINT` и `SIGTERM` устанавливают общий `asyncio.Event`. Каждая корутина проверяет событие в цикле, при ожидании ограничения скорости и во время backoff. Три источника запускаются через `asyncio.create_task`, затем ожидаются через `asyncio.gather`.

Ошибка одного источника возвращается как результат задачи. Другие источники получают возможность корректно завершить текущую работу. В терминал выводится одна понятная строка ошибки без большого Python traceback, а процесс завершается с кодом 1. При обычном `Ctrl+C` остановка не считается ошибкой.

В блоке `finally` всегда рассчитывается статистика документов в MongoDB и закрывается подключение. Поэтому после штатной остановки видно, сколько документов и байт накоплено по каждому источнику.

3.8 Механизм обновления

Документ считается требующим проверки, если `checked_at` старше семи суток. Обновление выполняется в несколько уровней:

1. arXiv сначала сравнивает `updated`, Microsoft Learn — `lastmod` sitemap;
2. при наличии ETag или Last-Modified отправляются `If-None-Match` и `If-Modified-Since`;
3. ответ 304 обновляет только `checked_at`;
4. для ответа 200 вычисляется SHA-256 HTML;
5. совпадающий хеш означает `unchanged`, отличающийся — `changed`;
6. новая версия HTML записывается только при изменении хеша.

Таким образом, повторная проверка не означает обязательную перезапись документа. Поля времени имеют разные значения: `first_fetched_at` — первая загрузка, `fetched_at` — загрузка текущей версии, `checked_at` — последняя проверка.

3.9 Возникшие проблемы и выбранные решения

Во время разработки возникли следующие проблемы.

Первая проблема — HTTP 500 от arXiv при большом глобальном смещении. Простое увеличение числа повторов не помогало, потому что ошибка была воспроизводимой. Решением стало разбиение выдачи по месяцам, где смещение снова начинается с нуля.

Вторая проблема — arXiv иногда возвращал 500 и для пакетной проверки метаданных. Чтобы один ID не блокировал весь пакет, пакет рекурсивно делится пополам, а единичный проблемный документ откладывается.

Третья проблема — при остановке во время backoff программа могла долго ждать. Обычный `asyncio.sleep` был заменён ожиданием, которое одновременно следит за таймером и событием остановки.

Четвёртая проблема — ошибка одного источника раньше приводила к длинному traceback после общей статистики. Обёртка `run_source` теперь записывает короткое сообщение, возвращает объект ошибки и позволяет завершить остальные источники.

Пятая проблема — первоначальный лог показывал только запуск источника. Я добавил табличные строки для каждого нового или изменённого документа: источник, статус, размер и сокращённый заголовок. Полный URL не выводится, поскольку он делает лог трудночитаемым и уже хранится в MongoDB.

При первом запуске Docker также возник конфликт порта 27017 с другим контейнером. После остановки прежнего контейнера сохранён стандартный локальный порт. Менять порт проекта не потребовалось.

Фрагмент фактического журнала работы имеет следующий вид:

```
SOURCE          | STATUS     |       SIZE | TITLE
wikipedia       | NEW        |  117.4 KiB | Uber Entertainment
microsoft_learn | NEW        |   82.0 KiB | Rename-Mg Beta Device App
Management...
wikipedia       | NEW        | 1039.1 KiB | Ubisoft
arxiv           | NEW        |  162.5 KiB | Structural Segmentation for
Brazilian...
```

4 План тестирования

Тестирование разделено на автоматические тесты отдельных механизмов и ручные интеграционные сценарии с MongoDB и реальными источниками.

Автоматические тесты запускаются командой:

```
uv run pytest -q tests/crawler
```

Автоматические тесты приведены в таблице 4.

Таблица 4 — Автоматические тесты поискового робота

№	Проверка	Ожидаемый результат	Итог
1	Нормализация URL	удалены fragment и tracking-параметры	пройден
2	Стабильность SHA-256	одинаковый HTML даёт одинаковый хеш	пройден
3	Сокращение заголовка лога	длинный заголовок не ломает таблицу	пройден
4	Разбор sitemap	извлекаются URL и `lastmod`	пройден
5	Разбор Atom arXiv	извлекаются ID и метаданные	пройден
6	Месячный запрос arXiv	нет глобального большого смещения	пройден
7	Корректный YAML	обязательные секции принимаются	пройден
8	Ошибочный YAML	неверная конфигурация отклоняется	пройден
9	Условный запрос 304	`checked_at` обновляется без HTML	пройден
10	Свежий документ	повторная загрузка не выполняется	пройден
11	Слишком большой ответ	документ пропущен, источник продолжен	пройден
12	Повторная проверка arXiv	сначала проверяются метаданные	пройден
13	Ошибка пакета arXiv	пакет делится, ID откладывается	пройден
14	Справка CLI	позиционный аргумент отображается	пройден
15	Chunked HTTP-ответ	все части ответа прочитаны	пройден
16	Остановка во время retry	ожидание немедленно прерывается	пройден
17	Штатная остановка	не записывается как ошибка	пройден
18	Ошибка источника	нет traceback, возвращается код ошибки	пройден

Ручные интеграционные сценарии приведены в таблице 5.

Таблица 5 — Интеграционные сценарии тестирования

№	Действие	Проверяемый результат	Итог
1	Запустить три источника	строки документов появляются параллельно	пройден
2	Нажать `Ctrl+C`	источники завершаются, выводится статистика	пройден
3	Запустить повторно	обход продолжается с MongoDB-состояния	пройден
4	Проверить индексы	пары source/ID и source/URL уникальны	пройден
5	Подождать временную ошибку	используются retry и backoff	пройден
6	Открыть документы MongoDB	есть URL, HTML, source и timestamps	пройден

5 Результаты

5.1 Результат автоматических тестов

На момент подготовки отчёта все тесты crawler проходят:

```
..................
[100%]
18 passed in 0.80s
```

Тесты не обращаются к внешним сайтам: HTTP-ответы и MongoStorage подменяются контролируемыми объектами. Благодаря этому временная недоступность arXiv не делает результат тестового плана случайным.

5.2 Накопленный корпус

Актуальная статистика коллекции `mai_ir.documents` приведена в таблице 6. Размер относится к сырому HTML в UTF-8.

Таблица 6 — Статистика сырых документов в MongoDB

Источник	Документов	Объём, MiB	Средний, KiB
arXiv	11 038	3 233,8	300,0
Microsoft Learn	80 899	5 562,8	70,4
Wikipedia	78 456	13 181,9	172,0
Всего	**170 393**	**21 978,4**	**132,1**

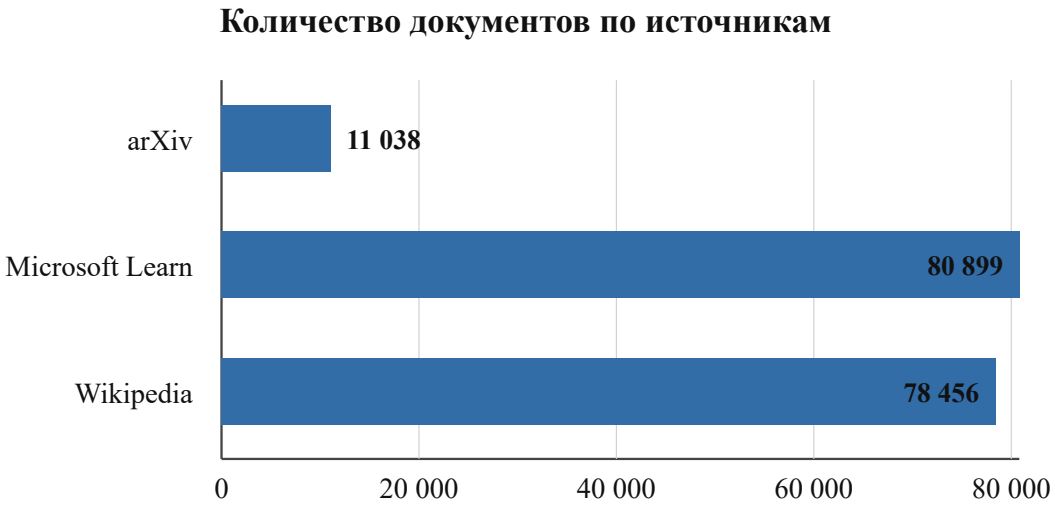


Рисунок 1 — Распределение документов по источникам

Распределение не задавалось искусственными квотами. Число документов определяется доступностью подходящих страниц и текущей позицией обхода. arXiv содержит меньше документов, но средний HTML значительно больше. Wikipedia занимает наибольший объём — около 12,9 GiB.

В очереди Wikipedia сохранено 88 828 задач: 81 933 завершены и 6 895 ожидают обработки. Состояние Microsoft Learn указывает на sitemap `powershell_en-us_2.xml` и позицию 22 037. Состояние arXiv указывает на февраль 2024 года, смещение 100 и элемент 17 ответа. Эти значения подтверждают, что позиция действительно хранится в MongoDB и доступна следующему запуску.

5.3 Пример сохранённого документа

Сокращённый документ Wikipedia выглядит следующим образом. Поле `raw_html` в примере опущено только из-за его размера; в MongoDB оно присутствует полностью.

```
{
  "source": "wikipedia",
  "source_id": "10000895",
  "url": "https://en.wikipedia.org/wiki/Incident_response_team",
  "title": "Incident response team",
  "raw_size": 130365,
  "content_hash": "ff128d393ac182f33669d2cca8dd9edd...",
  "metadata": {"page_id": "10000895"},
  "first_fetched_at": 1792663487,
  "fetched_at": 1792663487,
  "checked_at": 1792663487,
  "last_modified": "Wed, 07 Oct 2026 19:28:58 GMT"
}
```

Unix timestamp хранится целым числом, как требует задание. Отдельные поля первой загрузки, текущей версии и проверки позволяют отличить изменение HTML от обычной проверки актуальности.

5.4 Выполнение требований задания

Соответствие реализации пунктам lesson-02 приведено в таблице 7.

Таблица 7 — Соответствие реализации заданию lesson-02

Требование	Реализация	Итог
Один аргумент — YAML	`argparse` принимает `config`	выполнено

Требование	Реализация	Итог
Секции `db` и `logic`	`load_config` проверяет обе секции	выполнено
Нормализованный URL	`normalize_url` перед сохранением	выполнено
Сырой HTML	поле `raw_html`	выполнено
Название источника	поле `source`	выполнено
Unix timestamp	`first_fetched_at`, `fetched_at`, `checked_at`	выполнено
Продолжение после остановки	очередь и `crawl_state` в MongoDB	выполнено
Периодическая переобкачка	порог 7 суток и условные запросы	выполнено
Запись только изменений	ETag, Last-Modified и SHA-256	выполнено

Метрики P, DCG, NDCG и ERR для поискового робота не вычисляются: компонент не формирует ранжированную выдачу и не принимает пользовательские поисковые запросы. Его качество оценивается полнотой сохранения сырого документа, отсутствием дублей, корректностью продолжения и обновления, устойчивостью к сетевым ошибкам.

Общий README курса связывает оценку «отлично» с корпусом более одного миллиона документов. Текущие 170 393 документа превышают минимальный учебный масштаб в 30 000, но ещё не достигают миллиона. Архитектура не требует загрузки корпуса в оперативную память: очередь, состояния и документы читаются и записываются порциями, поэтому обход можно продолжать без изменения формата хранения.

6 Выводы

В работе я реализовал асинхронный поисковый робот для Wikipedia, arXiv HTML и Microsoft Learn. Программа принимает единственный путь к YAML-конфигурации, учитывает `robots.txt`, ограничивает частоту запросов и сохраняет сырой HTML с нормализованным URL, источником и Unix timestamps в MongoDB.

Главный результат работы — устойчивый возобновляемый обход. Wikipedia использует постоянную очередь с lease, arXiv сохраняет месяц и позицию Atom-выдачи, Microsoft Learn сохраняет sitemap и индекс URL. После `Ctrl+C` следующий запуск продолжает накопление корпуса без повторного старта.

Механизм обновления сочетает метаданные источника, условные HTTP-заголовки и SHA-256. Неизменившийся документ не перезаписывается, а

изменившийся сохраняется с новым `fetched_at`. Два уникальных индекса предотвращают дубли по ID и URL.

В MongoDB накоплено 170 393 документа общим объёмом около 21,5 GiB. Все 18 автоматических тестов проходят. Робот продолжает работу при слишком большом ответе или единичной проблеме документа; временные сетевые ошибки повторяются с backoff.

К недостаткам относятся зависимость от внешних API и структуры sitemap, сравнительно медленный arXiv из-за обязательной задержки, отсутствие распределённого запуска нескольких экземпляров и пока недостаточный для оценки «отлично» размер корпуса. Возможные улучшения — метрики скорости по источникам, наблюдаемость очередей, отдельная коллекция истории версий и увеличение корпуса до миллиона документов без повышения нагрузки на отдельный домен.

7 Использованные источники

1. Задание лабораторной работы № 2: https://github.com/toshunster/MAI-IR/blob/main/lesson-02/homework.md.
2. Общие требования курса: https://github.com/toshunster/MAI-IR/blob/main/README.md.
3. MediaWiki API: https://www.mediawiki.org/wiki/API:Main_page.
4. Руководство arXiv API: https://info.arxiv.org/help/api/user-manual.html.
5. Sitemap Microsoft Learn: https://learn.microsoft.com/_sitemaps/sitemapindex.xml.
6. Документация MongoDB: https://www.mongodb.com/docs/manual/.
7. Исходный код работы: https://github.com/dmitriikuleshov/mai-ir.